Chapter 11

Java Server Pages

CHAPTER OUTLINES

To discuss basic concepts of applets

- To compare JSP with Servlets and other server side technologies
- To demonstrate different tags used in JSP
- To explain different implicit objects provided by JSP
- To process HTML forms using JSP
- To access databases using JSP
- To handle exceptions and sessions using JSP
- To used java beans with JSP

11.1 Basic of Java Server Pages

Java Server Pages (JSP) is a server-side programming technology that enables the creation of dynamic, platform-independent method for building Web-based applications. JSP have access to the entire family of Java APIs, including the JDBC API to access enterprise databases. It can be thought of as an extension to servlet because it provides more functionality than servlet. A JSP page consists of HTML tags and JSP tags. The JSP pages are easier to maintain than servlet because we can separate designing and development.

Web developers write JSPs as text files that combine HTML or XHTML code, XML elements, and embedded JSP actions and commands. JSP tags can be used for a variety of purposes, such as retrieving information from a database or registering user preferences, accessing JavaBeans components, passing control between pages and sharing information between requests, pages etc.

11.1.1 Advantages of JSP

Following is the list of other advantages of using Java Server Pages over other technologies:

JSP vs ASP

The advantages of JSP are twofold

- The dynamic part is written in Java, not Visual Basic or other MS specific language, so it is more powerful and easier to use.
- It is portable to other operating systems and non-Microsoft Web servers.

JSP vs Servlets

There are many advantages of JSP over servlet. They are as follows:

- **Extension to Servlet:** JSP technology is the extension to servlet technology. We can use all the features of servlet in JSP. In addition to, we can use implicit objects, predefined tags, expression language and Custom tags in JSP, which makes JSP development easy.
- **Easy to maintain:** JSP can be easily managed because we can easily separate our business logic with presentation logic. In servlet technology, we mix our business logic with the presentation logic.
- **Fast Development:** If JSP page is modified, we don't need to recompile and redeploy the project. The servlet code needs to be updated and recompiled if we have to change the look and feel of the application.
- **Less code than Servlet:** In JSP, we can use a lot of tags such as action tags, custom tags etc. that reduces the code.

11.1.2 JSP Access Model

When a user goes to a web site that is developed by using JSP and tries to access JSP page by using a web browser, following steps are performed to process the request and return result to the user:

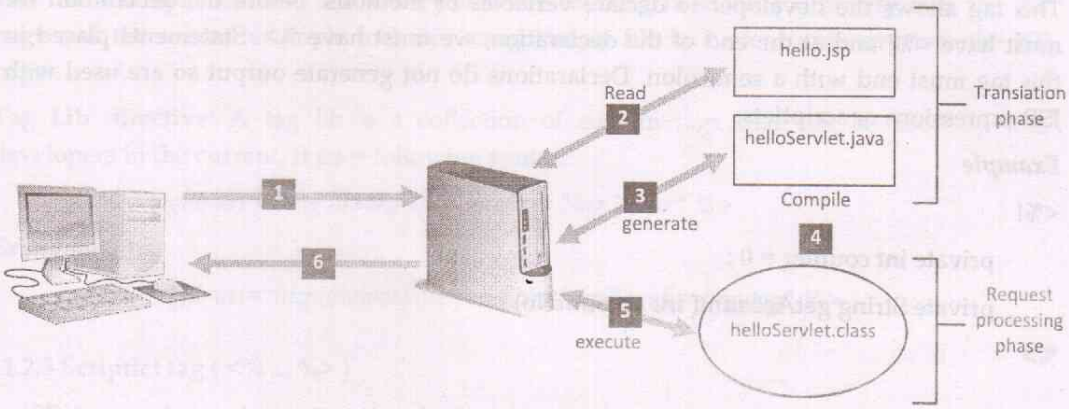


Figure 11.1 JSP Architecture.

1. User sends JSP request to the Web server through web browser to the web server with JSP engine.
2. The Web server recognizes that the file required is JSP file, therefore passes the JSP file to the JSP Engine.
3. If the JSP file has been called the first time, the JSP file is parsed, otherwise go to step 5. The JSP engine generates a special servlet file from the JSP file.
4. The servlet file is compiled into a executable class file. JSP engine then forwards the original request to a servlet engine and the servlet is instantiated.
5. The servlet engine loads the servlet class and executes it by calling init and service methods. During execution, the servlet produces an output in HTML format, which is passed to an HTTP response.
6. The web server forwards the HTTP response to browser in terms of static HTML content. Finally web browser handles the dynamically generated HTML page inside the HTTP response and results are displayed on the web browser.

Typically, the JSP engine checks to see whether a servlet for a JSP file already exists and whether the modification date on the JSP is older than the servlet? If the JSP is older than its generated servlet, the JSP container assumes that the JSP hasn't changed and that servlet file is not generated. This makes the process more efficient than with other scripting languages (such as PHP) and therefore faster.

11.2 JSP Syntax

There are five main tags that all together form JSP syntax. These tags are: Declaration tags, Expression tags, Directive tags, Scriptlet tags, and Action tags.

11.2.1 Declaration tag (<%! %>)

This tag allows the developer to declare variables or methods. Before the declaration we must have <%! and at the end of the declaration, we must have %>. Statements placed in this tag must end with a semicolon. Declarations do not generate output so are used with JSP expressions or scriptlets.

Example

```
<%!
    private int counter = 0 ;
    private String getAccount( int accountNo) ;
%>
```

11.2.2 Expression tag (<%= %>)

This tag allows the developer to embed any Java expression. It is the short hand form of statement "out.println()" written in servlets. A semicolon should not appear at the end of the statements wriiten inside the tag. For example, to show the current date and time, we can write

Date : <%= new java.util.Date() %>

11.2.2 Directive tag (<%@ directive ... %>)

It is a JSP directive that gives special information about the page to the JSP Engine. It usually has the following form: <%@ directive attribute="value" %>. There are three main types of directives:

- Page Directive - processing information for this page.
- Include Directive- files to be included.
- Tag library Directive- tag library to be used in this page.

Directives do not produce any visible output when the page is requested but change the way the JSP Engine processes the page. For example, we can make session data unavailable to a page by setting a page directive (session) to false.

Page directive: This directive has 11 optional attributes that provide the JSP Engine with special processing information. Some of the attributes are:

<%@ page language = "java" %>

<%@ page extends = "com.taglib... %>

<%@ page import = "java.util." %>*

Include Directive: It allows a JSP developer to include contents of a file inside another. File is included during translation phase. Examples are given below:

<%@ include file = "include/privacy.html" %>

<%@ include file = "navigation.jsp" %>

The first example includes the *privacy.html* file residing in include directory into the JSP page and the second example includes *navigation.jsp* file in current directory into the JSP page

Tag Lib directive: A tag lib is a collection of custom tags that can be used by web developers in the current. It uses following syntax:

<%@ taglib uri = "tag library URI" prefix = "tag Prefix" %>

Example

<%@ taglib uri="http://www.javatpoint.com/tags" prefix="mytag" %>

11.2.3 Scriptlet tag (<% ... %>)

In JSP, java code can be written inside the jsp page using the scriptlet tag. Code is written between <% and %> tags. This code can access any variable or bean declared. For example, to print a variable, we can write scriptlet as below:

```
<%
    String username = "Suman" ;
    out.println ( username ) ;
%>
```

11.2.4 Action tag

The action tags are used to control the flow between pages and to use Java Bean. Some of the action tags supported by JSP are: jsp:forward, jsp:include, jsp:param etc. An example of action tag that forwards to another JSP page is given below:

<jsp:forward page="printdate.jsp" />

11.2.5 JSP Comment

JSP comments marks text or statements that the JSP container should ignore. A JSP comment is useful when we want to hide or "comment out" part of our JSP page. Following is the syntax of JSP comments:

<%-- This is JSP comment --%>

11.3 JSP Implicit Objects

There are several objects that are automatically available in JSP called implicit objects. These objects are available to developers in each page and developer can call them directly

without being explicitly declared. JSP Implicit Objects are also called pre-defined variables. There are nine implicit objects defined in JSP container. These objects are described below:

- **out object-** This object is used for writing any data to the buffer. It is the object of PrintWriter class.
- **request object-** This is the object of HttpServletRequest class. This object is normally used in looking up parameter values and cookies.
- **response object-** This is the object of HttpServletResponse class. Through this object the JSP programmer can add new cookies or date stamps, HTTP status codes etc.
- **config object-** The config object is an instantiation of ServletConfig class. This object can be used to get initialization parameter for a particular JSP page.
- **exception object-** exception is an implicit object of Throwable class. This object can be used to print the exception.
- **Application object-** It is an instance of ServletContext class and is created only once by the web container when application or project is deployed on the server. This object can be used to get initialization parameter from configuration file (web.xml).
- **Session object-** session is an implicit object of type HttpSession. The Java developer can use this object to set,get or remove attribute or to get session information.
- **Page object-** page is an implicit object of type Object class. This object is assigned to the reference of auto generated servlet class.
- **pageContext object-** pageContext is an implicit object of type PageContext class. The pageContext object can be used to set, get or remove attribute from object scopes.

11.4 Scope of JSP Objects

The availability of a JSP object for use from a particular place of the application is defined as the scope of that JSP object. Every object created in a JSP page will have a scope. Object scope in JSP is segregated into four parts and they are page, request, session and application.

- **Page Scope-** page scope means, the JSP object can be accessed only from within the same page where it was created. JSP implicit objects out, exception, response, pageContext, config and page have page scope.

```
//Example of JSP Page Scope
<jsp:useBean id="employee" class="EmployeeBean" scope="page" />
```

- **Request Scope-** A JSP object created using the request scope can be accessed from any pages that serves that request. More than one page can serve a single request. Implicit object request has the request scope.

```
//Example of JSP Request Scope
```

<jsp:useBean id="employee" class="EmployeeBean" scope="request" />

- **Session Scope-** session scope means, the JSP object is accessible from pages that belong to the same session from where it was created. Implicit object session has the session scope.

  //Example of JSP Session Scope

  <jsp:useBean id="employee" class="EmployeeBean" scope="session" />

- **Application Scope-** A JSP object created using the application scope can be accessed from any pages across the application. The. Implicit object application has the application scope.

  //Example of JSP application Scope

  <jsp:useBean id="employee" class="EmployeeBean" scope="application" />

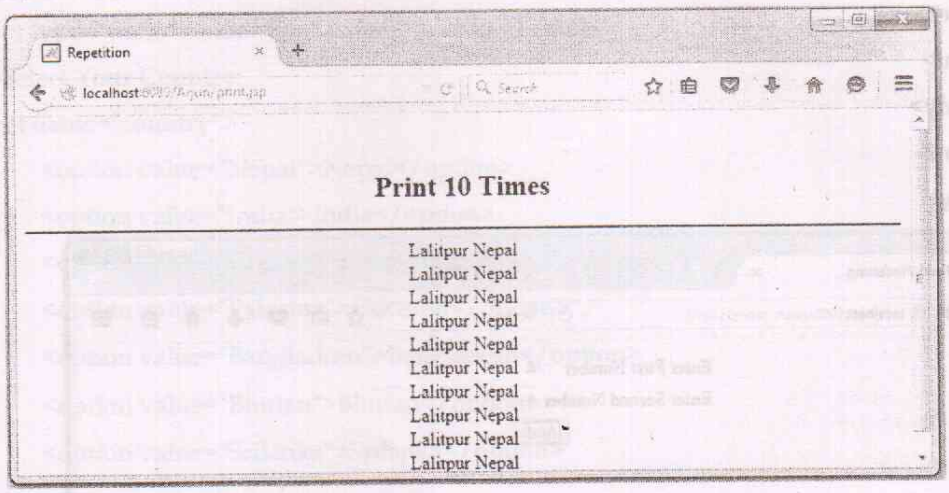
//Example---*First JSP Example---first.jsp File*

```
<html>
<head>
<title>First JSP Example</title>
</head>
<body>
<font size=3>
<br><br><center><h2>

<%-- Display by using expression tag --%>
<%= "First JSP Example" %>
<hr color=blue size=3>

<%-- Display by using scriptlet tag --%>
<%
out.println("Hello WebApp developer.....Welcome to JSP");
%>
<h2>
<center>
</font>
</body>
</html>
```

Output

```
//Second Example-Printing Message 10 times
//print.jsp File
<html>
<head>
<title>Repetition</title>
</head>
<body>
<font size=3>
<br><center><h2>
<%-- Display by using expression tag --%>
<%= "Print 10 Times" %>
</h2>
<hr color=blue size=3>
<%-- Display by using scriptlet tag --%>
<%
        for (int i=0;i<10;i++)
        out.println("Lalitpur Nepal<br>");
%>
<center>
</font>
</body>
</html>
```

Output



11.5 Form Processing in JSP

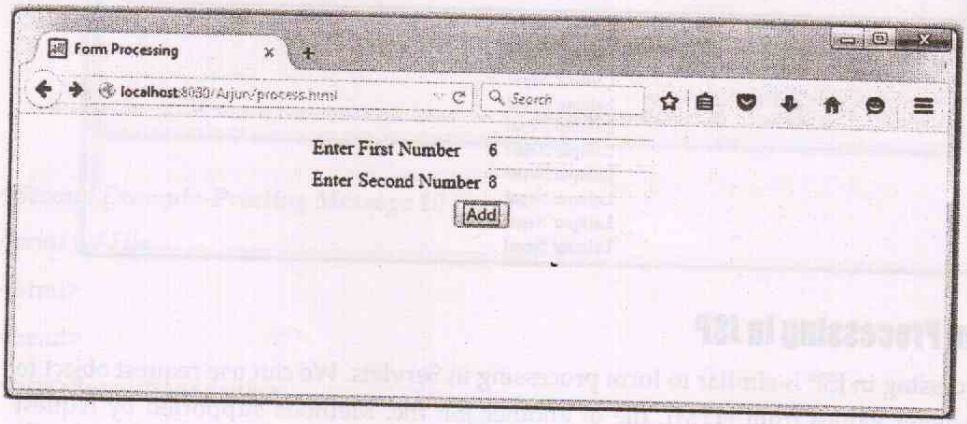
Form processing in JSP is similar to form processing in Servlets. We can use request object to read parameter values from HTML file or another JSP file. Methods supported by request object are similar to methods used in dervlets.

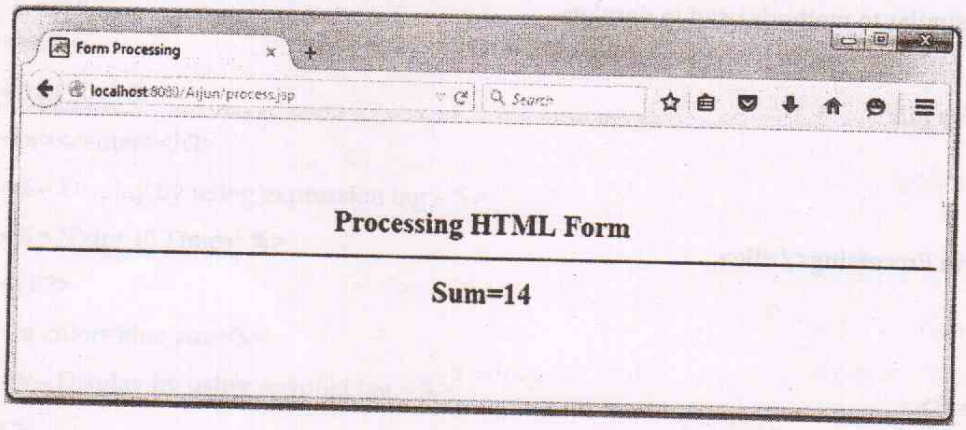
//process.jsp File

```
<html>
<head>
<title>Form Processing</title>
</head>
<body>
<font size=3>
<br><br><center><h2>
Processing HTML Form
<hr color=red size=3>
<%
        int x=Integer.parseInt(request.getParameter("first"));
        int y=Integer.parseInt(request.getParameter("second"));
        int z=x+y;
        out.println("Sum="+z);
%>
```

```
<h2>
<center>
</font>
</body>
</html>
```

Output





Example with more elements

//myform.html File

```
<html>
<body>
<h2>Please Tell me about yourself</h2>
<hr color=red size=3>
<form action="myfrom.jsp" method="post">
Name: <input type="text" name="firstName">
<input type="text" name="lastName"><br>
```

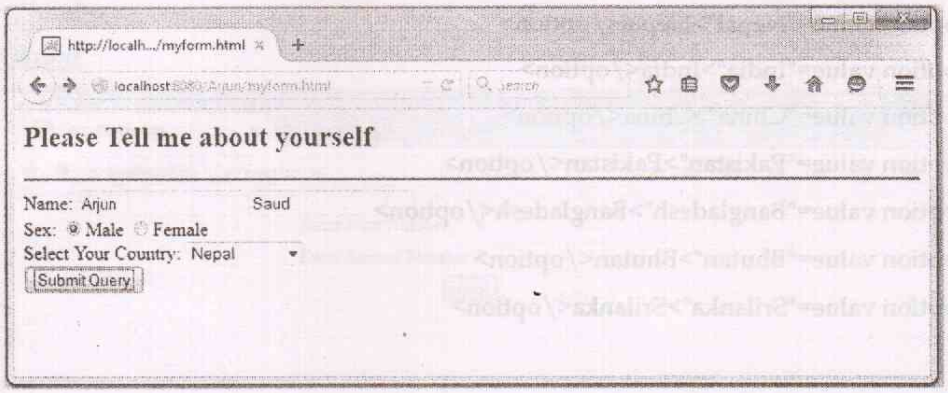
```
Sex: <input type="radio" checked name="sex" value="male">Male
<input type="radio" name="sex" value="female">Female
<br>Select Your Country:
<select name="country">
        <option value="Nepal">Nepal</option>
        <option value="India">India</option>
        <option value="China">China</option>
        <option value="Pakistan">Pakistan</option>
        <option value="Bangladesh">Bangladesh</option>
        <option value="Bhutan">Bhutan</option>
        <option value="Srilanka">Srilanka</option>
</select>
<br>
<input type="submit">
</form>
</body>
</html>
```

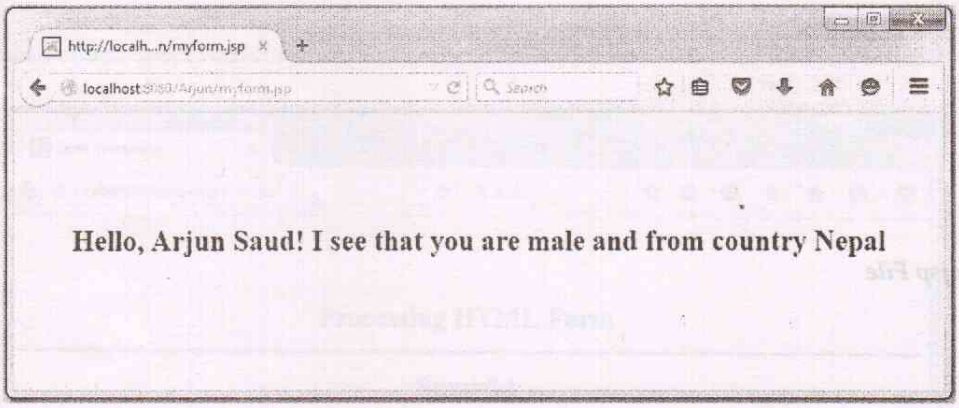
//myform.jsp File

```
<html>
<body>
<%
        String fn = request.getParameter("firstName");
        String ln = request.getParameter("lastName");
        String sex = request.getParameter("sex");
        String country = request.getParameter("country");
%>
<br><br><br>
<center>
<h2>Hello, <%=fn+" "+ln+"!"%>
I see that you are <%=sex%> and from country
<%=country%>
</h2>
```

```
</center>
</body>
</html>
```

Output





11.6 Database Access with JSP

We also access database by using JSP. All JDBC APIs can be accessed from JSP. We have put all codes related to database access within the expression tag.

//addbook.html File

```
<html>
<head>
<Title> E-book Store </Title>
</head>
<body>
<br><Center>
<h2>Adding New Book</h2>
<hr color=blue size=3>
<form action="addbook.jsp" method="post">
```

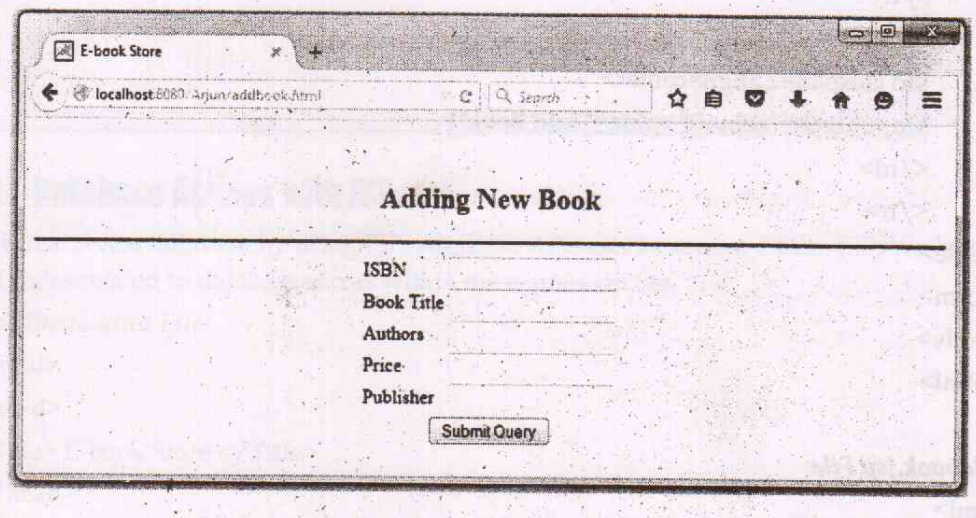
```
<table>
    <tr>
    <td> ISBN </td>
    <td><input type=text name=ISBN></td>
    </tr>
    <tr>
    <td> Book Title </td>
    <td><input type=text name=title></td>
    </tr>
    <tr>
    <td> Authors</td>
    <td><input type=text name=authors></td>
    </tr>
    <tr>
    <td> Price </td>
    <td><input type=text name=price></td>
    </tr>
    <tr>
    <td> Publisher </td>
    <td><input type=text name=pub></td>
    </tr>
    <tr>
    <td colspan=2 align=center>
    <input type="submit" value="Add Book">
    </td>
    </tr>
</table>
</form>
</body>
</html>
```

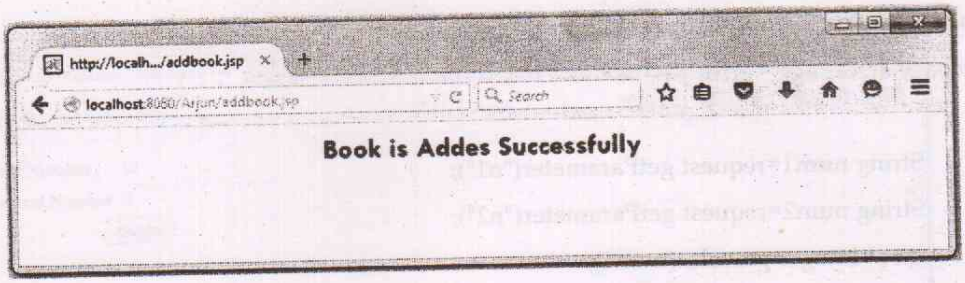
//addbook.jsp File

```
<html>
<body>
<%
    String ISBN = request.getParameter("ISBN");
    String title = request.getParameter("title");
```

```
        String auth = request.getParameter("authors");
        int price = Integer.parseInt(request.getParameter("price"));
        String pub=request.getParameter("pub");
%>
<h2><center>
<font face="Tw Cen MT" color=blue>
<%@ page import = "java.sql.*" %>
<%
 Connection con =  DriverManager.getConnection("jdbc:mysql://localhost/ebook", "root",
"arjun");
 Statement stmt = conn.createStatement();
 String sql ="insert into books values ('"+ISBN+ "','"+title+ "','"+auth+"'," +price+",'"
+pub+"')";
stmt.executeUpdate(sql);
out.println("Book is Addes Successfully");
%>
</font><center><h2>
</body>
</html>
```

Output





11.7 Exception Handling in JSP

The exception is normally an object that is thrown at runtime. Exception Handling is the process to handle the runtime errors. There may occur exception at any time in our web application. So handling exceptions is a safer side for the web developer. JSP gives us an option to specify Error Page for each JSP. Whenever the page throws an exception, the JSP container automatically invokes the error page. As we know, exception is an implicit object of type Throwable class. This object can be used to print the exception. But it can only be used in error pages. To set up an error page, use the <%@ page errorPage="xxx.jsp" %> directive.

Example

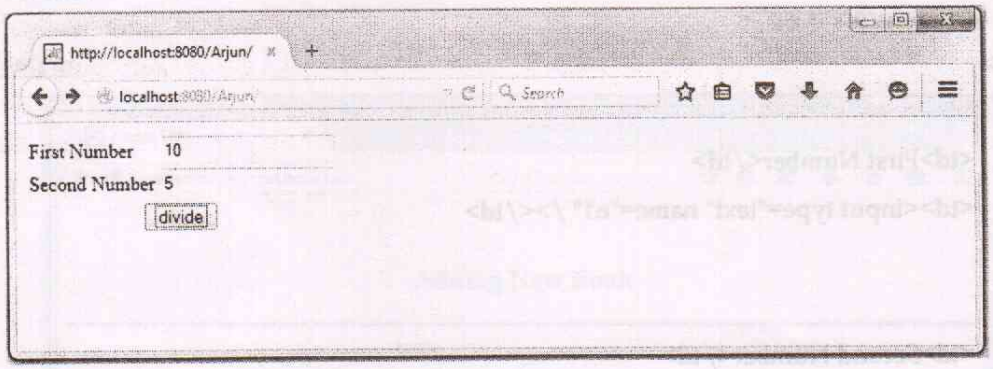
//index.jsp File

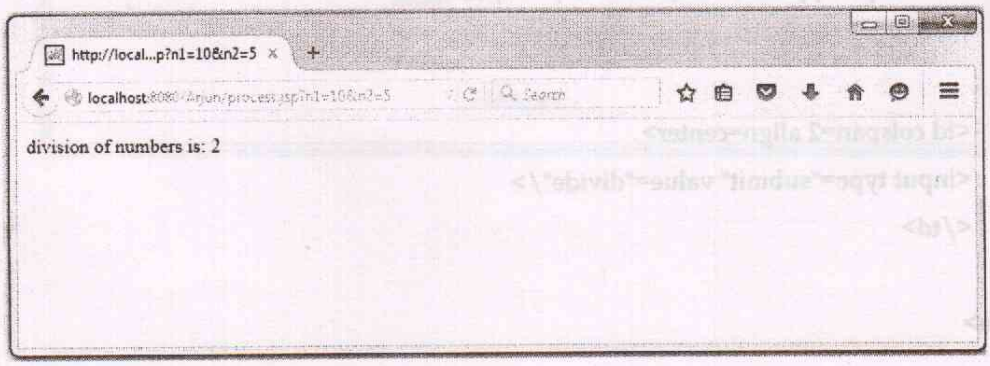
```
<form action="process.jsp">
<table>
<tr>
        <td>First Number</td>
        <td><input type="text" name="n1" /></td>
</tr>
<tr>
        <td>Second Number</td>
        <td><input type="text" name="n2" /> </td>
</tr>
<tr>
        <td colspan=2 align=center>
        <input type="submit" value="divide"/>
        </td>
</tr>
</form>
```

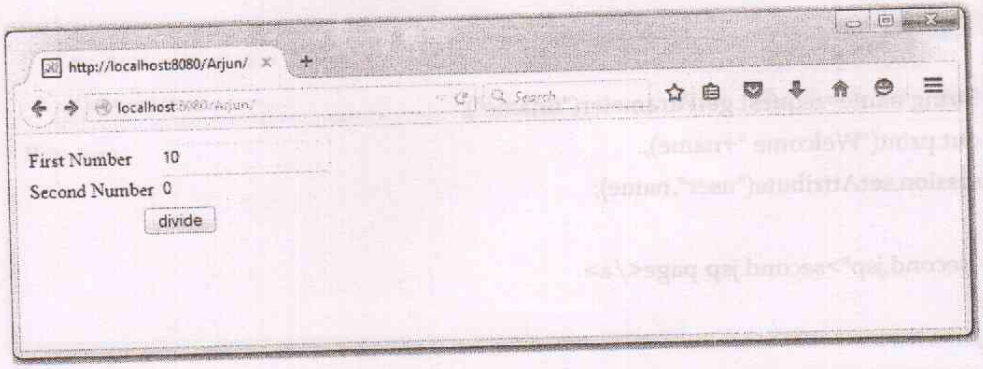
```
//process.jsp File
<%@ page errorPage="error.jsp" %>
<%
        String num1=request.getParameter("n1");
        String num2=request.getParameter("n2");
        int a=Integer.parseInt(num1);
        int b=Integer.parseInt(num2);
        int c=a/b;
        out.print("division of numbers is: "+c);
%>
```

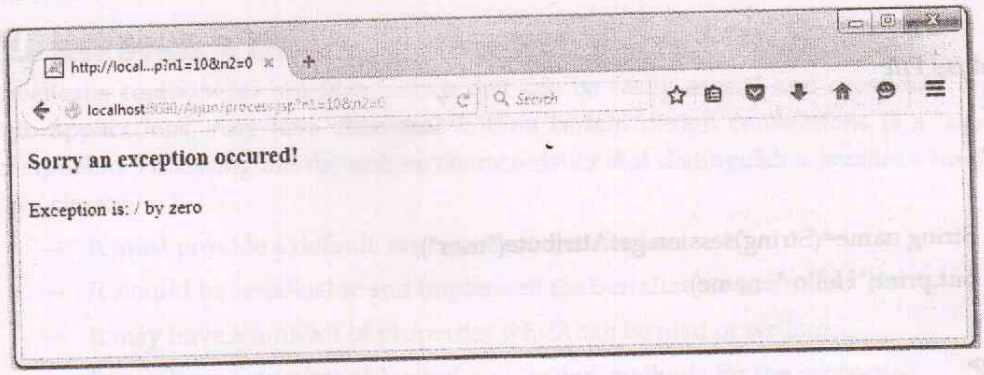
```
//error.jsp
<%@ page isErrorPage="true" %>
<h3>Sorry an exception occured!</h3>
Exception is: <%= exception.getMessage() %>
```

Output









11.8 JSP Session Tracking

In JSP, session is an implicit object of type HttpSession.The Java developer can use this object to set,get or remove attribute or to get session information. By default, JSPs have session tracking enabled and a new HttpSession object is instantiated for each new client automatically. Disabling session tracking requires explicitly turning it off by setting the page directive session attribute to false as follows:

```
<%@ page session="false" %>
```

Example

//index.html File

```
<html>
<body>
    <form action="welcome.jsp">
    <input type="text" name="uname">
    <input type="submit" value="go"><br/>
</form>
</body>
</html>
```

// welcome.jsp File

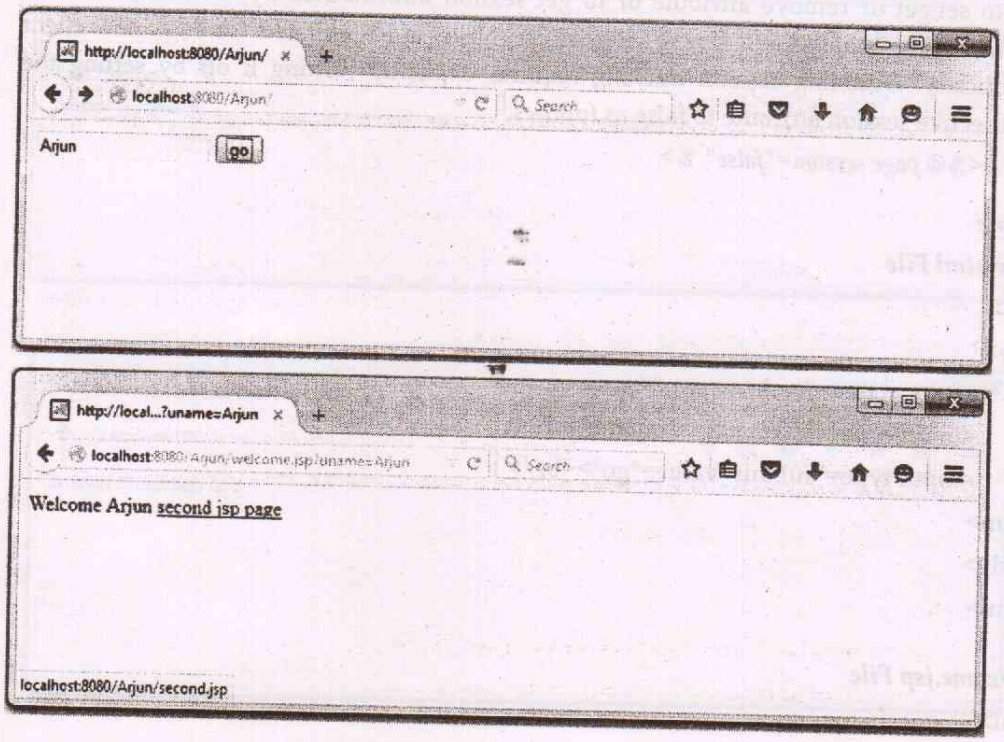
```
<html>
```

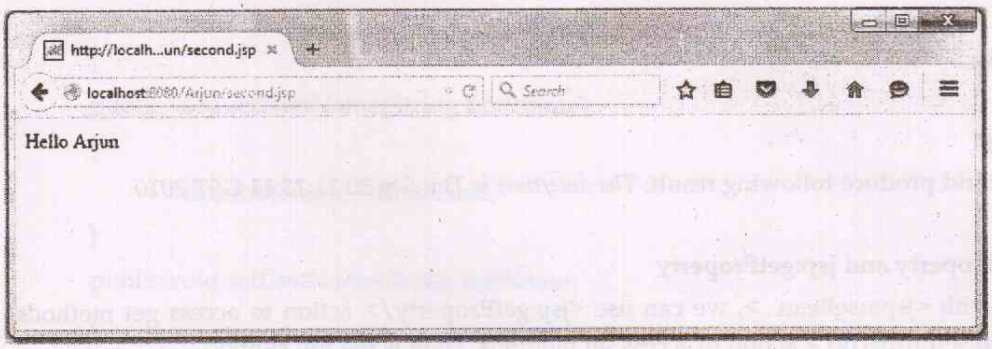
```
<body>
<%
        String name=request.getParameter("uname");
        out.print("Welcome "+name);
        session.setAttribute("user",name);
%>
<a href="second.jsp">second jsp page</a>
</body>
</html>
```

//second.jsp File

```
<html>
<body>
<%
        String name=(String)session.getAttribute("user");
        out.print("Hello "+name);
%>
</body>
</html>
```

Output





11.9 Java Beans in JSP

JavaBeans components are Java classes that can be easily reused and composed together into applications. Any Java class that follows certain design conventions is a JavaBeans component. Following are the unique characteristics that distinguish a JavaBean from other Java classes:

→ It must provide a default, no-argument constructor.

→ It should be serializable and implement the Serializable interface.

→ It may have a number of properties which can be read or written.

→ It may have a number of "getter" and "setter" methods for the properties.

JavaServer Pages technology directly supports using JavaBeans components with standard JSP language elements. We can easily create and initialize beans and get and set the values of their properties.

jsp:useBean

The **useBean** action declares a JavaBean for use in a JSP. Once declared, the bean becomes a scripting variable that can be accessed by both scripting elements and other custom tags used in the JSP. The full syntax for the useBean tag is as follows:

<jsp: useBean id="beans name" class="class name" scope="beans scope" />

Here values for the scope attribute could be page, request, session or application based on our requirement. The value of the **id** attribute may be any value as a long as it is a unique name among other useBean declarations in the same JSP.

Example

```
<html>
    <head>
        <title>useBean Example</title>
    </head>
    <body>
        <jsp:useBean id="date" class="java.util.Date" />
        <p>The date/time is <%= date %>
```

```
    </body>
</html>
```

Output

This would produce following result: *The date/time is Thu Sep 30 11:18:11 GST 2010*

jsp:setProperty and jsp:getProperty

Along with <jsp:useBean...>, we can use <jsp:getProperty/> action to access get methods and <jsp:setProperty/> action to access set methods. Here is the full syntax:

```
<jsp:useBean id="beans id" class="class name" scope="beans scope">
  <jsp:setProperty name="beans id" property="property name"
            value="value"/>
  <jsp:getProperty name="beans id" property="property name"/>
  ...........
</jsp:useBean>
```

The name attribute references the id of a JavaBean previously introduced to the JSP by the useBean action. The property attribute is the name of the get or set methods that should be invoked.

Example

```
//Student.java File
public class Student implements java.io.Serializable
{
    private String firstName = null;
    private String lastName = null;
    private int age = 0;
    public Student() {   }
    public String getFirstName()
    {
        return firstName;
    }
    public String getLastName()
    {
        return lastName;
    }
    public int getAge()
    {
```

```
            return age;
        }
        public void setFirstName(String firstName)
        {
            this.firstName = firstName;
        }
        public void setLastName(String lastName)
        {
            this.lastName = lastName;
        }
        public void setAge(Integer age)
        {
            this.age = age;
        }
}
//Student.jsp File
<html>
<head>
<title>get and set properties Example</ title>
</head>
<body
        <jsp:useBean id="st" class="Student">
            <jsp:setProperty name="st" property="firstName" value="Aagaman"/>
            <jsp:setProperty name="st" property="lastName" value="Saud"/>
            <jsp:setProperty name="st" property="age" value="5"/>
        </jsp:useBean>
        <p>Student First Name:
        <jsp:getProperty name="st" property="firstName"/>
        </p>
        <p>Student Last Name:
        <jsp:getProperty name="st" property="lastName"/>
        </p>
        <p>Student Age:
        <jsp:getProperty name="st" property="age"/>
```

```
    </p>
</body>
</html>
```

Output

This would produce following result:

Note: *Make Students.class available in CLASSPATH and try to access above JSP.*

Exercise

1. How JSP is different from Servlets? Explain advantages of JSP over other server side scripting languages.
2. Explain JSP access model with suitable diagram.
3. What is meant by implicit objects? Explain different implicit objects provided by JSP briefly.
4. Write a JSP script that reads value from a textbox and identifies whether it is odd or even.
5. List the different tags available in JSP. Explain each of the tag with brief description and syntax.
6. What is meant by scope of objects? Explain different scopes of JSP objects briefly with example.
7. Write a JSP script that that demonstrates use of select and insert operations with suitable example.
8. How JSP can be used for enabling and disbling session? Explain with suitable JSP script.
9. How exceptions can be handled in JSP scripts? Explain with suitable JSP script.
10. What is meant by Java Bean? How it can be used in JSP scripts? Explain with suitable example.

❑❑❑